

EXHALE: User Manual

Scope. This manual describes how to set up, run, and analyze models with EXHALE, the in-house extension of the ATES (ATmospheric EScape) code to trace metals, metal-line cooling, a runtime opacity dispatcher, the Roche tidal potential, and excited-hydrogen / Ly α physics. It documents every input parameter (with a brief physical meaning), every output file (column by column, with units), the analysis tools, and a set of worked examples. The detailed physics derivations live in the companion notes (Update_EXHALE_early_phase.tex, transmission_spectrum.tex, Update_EXHALE.tex, cooling_formulas.tex — the complete reference for every analytic cooling coefficient — EXHALE_BC_and_IC.tex — the boundary- and initial-condition reference — and wind_ae_solver.tex — the bundled Wind-AE steady-wind solver used by IC mode: windae); this document is the practical user guide.

Contents

1. Introduction	2
1.1 What the code does	2
1.2 The lower boundary, radii, and normalization	3
1.3 The gravitational potential	3
1.4 Workflow	4
2. Installation and running	4
2.1 Requirements	4
2.2 Build	4
2.3 Running	5
2.4 Convergence control	5
2.5 The steady-state Newton solver (Solver: Newton)	5
2.6 Environment-variable hooks (diagnostics and expert runs)	6
2.7 Metal cooling on/off and the comparison script	7
2.8 Wind-AE warm-start initial conditions	7
3. Input files	9
3.1 input.inp — the main parameter file	9
3.2 metals.inp — trace-metal abundances	15
3.3 opacity.inp — opacity model (optional)	15
3.4 Excited hydrogen and the Ly α pumping field	16
3.5 Spectral coverage for metal photoionization	16
3.6 Diffusive separation of helium and metals	18
3.7 Lower-atmosphere connection (Tier 1–3)	18
3.8 Obtaining VULCAN and FastChem	20

4. Output files	20
4.1 Hydro_ioniz.txt / Hydro_ioniz_adv.txt	21
4.2 Ion_species.txt / Ion_species_adv.txt	22
4.3 Cooling_breakdown.txt	22
4.4 Excited_H.txt	22
4.5 ATES.out and the mass-loss rate	22
5. Analysis tools	22
5.1 EXHALE_plots.py — live profile viewer	22
5.2 TPM.py — transmission spectra	23
5.3 exhale_io.py — named, physical-unit loaders	23
5.4 exhale_to_lart.py — LaRT Ly α input from an EXHALE run	26
6. Worked examples	26
6.1 A first run: a generic hot Jupiter	26
6.2 Turning physics on and off	27
6.3 Ready-made example configurations (examples/)	29
7. Comparison studies	29
7.1 Effect of metal-line cooling (metals on vs. off)	29
7.2 Spherical vs. Roche (RLOF) and the mass-loss rate	29
7.3 Transmission spectra	31
8. Atomic and molecular data	31

1. Introduction

1.1 What the code does

EXHALE solves the time-dependent, spherically-symmetric (1-D) radiation–hydrodynamics of an irradiated planetary upper atmosphere and the hydrodynamic wind it drives (“atmospheric escape”). It integrates the Euler equations for mass, momentum, and energy of a single fluid,

$$\partial_t \rho + \frac{1}{r^2} \partial_r (r^2 \rho v) = 0, \quad \partial_t (\rho v) + \frac{1}{r^2} \partial_r (r^2 \rho v^2) + \partial_r p = -\rho \partial_r \Phi,$$

$$\partial_t E + \frac{1}{r^2} \partial_r (r^2 v (E + p)) = -\rho v \partial_r \Phi + \mathcal{H} - \mathcal{C},$$

to a steady state, where Φ is the gravitational (optionally Roche) potential (§1.3), $\mathcal{H}$ is the XUV photoionization heating and $\mathcal{C}$ the radiative cooling. Coupled to the hydrodynamics, at every step it solves the *ionization equilibrium* of H, He, and the trace metals (C, N, O, Mg, Si, Ca, Na, K, S, Fe) with a Newton (MINPACK) solver, including photoionization, collisional ionization, radiative + dielectronic recombination, and charge exchange. The converged run yields the radial profiles of density, velocity, temperature, and the ionization state, plus the steady-state *mass-loss*

rate $\dot{M}$. A post-processing pass adds advection to the ionization balance, and the Python tool `TPM.py` turns a converged model into transit *transmission spectra* (He 10830, Ly α , H α /H β , and the metal resonance lines Mg II, Ca II, Na I).

1.2 The lower boundary, radii, and normalization

The computational domain starts at a **lower boundary** placed at the 1 μ bar pressure level, which defines $r = 1$ and the base radius R_p (the input `Planet radius`). The base number density n_0 and temperature T_0 (the equilibrium temperature) are set there. Internally the code is non-dimensional: lengths in R_p , velocities in $v_0 = \sqrt{k_B T_0 / \mu}$, densities in n_0 , etc. *All output files are written back in cgs* (see §4. for the exact normalization of each column). The key dimensionless number is the *Jeans / escape parameter* $b_0 = GM_p \mu / (k_B T_0 R_p)$, the depth of the gravitational well in units of the thermal energy; small b_0 means an easily-escaping (or tidally unbound) atmosphere.

1.3 The gravitational potential

The potential Φ in the momentum and energy equations is selected by `Domain mode` (Table 3). The code writes it in the dimensionless units introduced above (lengths in R_p , potential in $v_0^2 = k_B T_0 / \mu$), so it is fixed by just three numbers: the Jeans parameter $b_0 = GM_p \mu / (k_B T_0 R_p)$, the stellar-to-planet mass ratio $q = M_\star / M_p$, and the orbital separation $\tilde{a} = a / R_p$.

Spherical mode (Domain mode: Spherical). Only the planet's own gravity is kept, so the domain can extend far past the Hill radius:

$$\phi(r) = -\frac{b_0}{r}.$$

Roche mode (default). The full co-rotating Roche potential, evaluated along the star–planet (substellar) line $x = r$, adds the stellar tidal attraction and the centrifugal term of the synchronously-rotating frame:

$$\phi(r) = -\frac{b_0}{r} - \frac{b_0 q}{\tilde{a} - r} - \frac{b_0 (1+q)}{2\tilde{a}^3} \left(\frac{q\tilde{a}}{1+q} - r \right)^2,$$

where the second term is the star's gravity at distance $\tilde{a} - r$ and the third is the centrifugal potential about the center of mass at $x_{\text{cm}} = q\tilde{a} / (1+q)$ with orbital frequency $\Omega^2 = G(M_p + M_\star) / \tilde{a}^3$ (synchronous rotation). The effective gravity used by the hydrodynamics is $g(r) = -d\phi/dr$, with

$$\frac{d\phi}{dr} = \frac{b_0}{r^2} - \frac{b_0 q}{(\tilde{a} - r)^2} + \frac{b_0 (1+q)}{\tilde{a}^3} \left(\frac{q\tilde{a}}{1+q} - r \right).$$

The inner Lagrange point L_1 is the root of $d\phi/dr = 0$, where planetary gravity, the stellar tide, and the centrifugal force cancel; in Roche mode the outer boundary is placed at this L_1 radius by default, beyond which gas is no longer bound to the planet.

Roche mode with an extended outer boundary (Outer radius in Roche mode). By default Roche mode truncates at L_1 . Supplying an explicit `Outer radius [R_p]: > 1 without Domain mode: Spherical` now keeps the full tidal Roche potential above *and* pushes the outer boundary out to the requested radius (e.g. $10 R_p$), reproducing the extended-domain treatment of Yan et al. (2022) / Huang et al. (2023), whose escape models retain the tidal force but integrate well past L_1 . This differs from `Domain mode: Spherical`, which also extends the domain but *drops* the tidal and

centrifugal terms (pure $-b_0/r$). Two caveats apply. First, beyond L_1 the Roche potential is past its maximum, so the net force is outward and the 1-D radial flow there is a spherical approximation to what is physically an L_1 funnel — the same approximation Yan/Huang make. Second, the cold hydrostatic IC is invalid past L_1 ; use IC mode: auto (or transonic) so the solver seeds a transonic wind through the interior sonic point. The tidal singularity of the $-b_0 q/(\tilde{a}-r)$ term sits at the star ($r = \tilde{a} \gg 10$), so a $10R_p$ domain is numerically safe.

Full 3-D form (transmission post-processing). The 3-D Roche-equipotential reconstruction in `TPM.py` / `roche_recon.py` (§5.2) needs the potential off-axis as well. With the substellar direction $\hat{x}$ pointing toward the star and (y, z) the sky plane,

$$\phi(x, y, z) = -\frac{b_0}{\sqrt{x^2 + y^2 + z^2}} - \frac{b_0 q}{\sqrt{(\tilde{a}-x)^2 + y^2 + z^2}} - \frac{b_0(1+q)}{2\tilde{a}^3} \left[(x_{\text{cm}} - x)^2 + y^2 \right],$$

which reduces to the substellar $\phi(r)$ above on the $+x$ axis. The centrifugal term involves only the rotation-plane coordinates (x, y) , not z , so surfaces of constant ϕ are triaxial (most extended toward L_1 , least along the spin axis z).

1.4 Workflow

A model run consists of:

1. Prepare `input.inp` (planet/star parameters, numerics, physics switches) — by hand or via the GUI; optionally `metals.inp` (trace metal abundances) and `opacity.inp` (opacity model).
2. Build and run `./EXHALE.x`; it reads the input files from the current directory and writes profiles to `output/`.
3. Inspect with `EXHALE_plots.py` (live profiles) and the example notebook; compute transmission spectra with `TPM.py`.

2. Installation and running

2.1 Requirements

A Fortran compiler (gfortran by default; Intel ifort/ifx supported), make, and — for the GUI and analysis — Python 3 with tkinter, numpy, scipy, and matplotlib.

2.2 Build

EXHALE uses an incremental Makefile; module dependencies are auto-generated, so only edited sources and their dependents recompile.

```
make                # build ./EXHALE.x with gfortran
make FC=ifort       # Intel ifort (or FC=ifx for the LLVM compiler)
make clean          # remove .o/.mod (keeps EXHALE.x)
make distclean      # wipe build/ and EXHALE.x
```

2.3 Running

The driver script `run_EXHALE.sh` (1) launches a Python/Tk GUI that writes `input.inp`, (2) builds, and (3) runs `./EXHALE.x`:

```
chmod +x run_EXHALE.sh
./run_EXHALE.sh          # gfortran (GUI -> build -> run)
./run_EXHALE.sh --ifort  # Intel ifort  (--ifx for ifx)
```

You may equally skip the GUI: place a hand-written `input.inp` (and optional `metals.inp/opacity.inp`) in a run directory, ensure an output/ subdirectory exists, and run `./EXHALE.x` there. Each modelled planet typically lives in its own subdirectory holding its `input.inp`, its `EXHALE.x`, and its output*/ folders. Progress (iteration counter, the momentum-variation du , and the time-derivative norm dtu) is printed to the terminal; the run stops when du or dtu falls below its convergence threshold.

2.4 Convergence control

The time loop stops when *any* of the following criteria is met (defaults in `src/modules/init/parameters.f90`; most are runtime-settable, Table 3):

Parameter	Default	Stop condition
<code>du_th</code>	1×10^{-3}	momentum non-uniformity $du < du_{th}$
<code>dtu_th</code>	1×10^{-8}	time-derivative ∞ -norm $dtu < dtu_{th}$
<code>stall_tol</code> , <code>N_stall</code>	10^{-6} , 2000	du on a plateau for <code>N_stall</code> steps
<code>count_max</code>	10^6	hard iteration cap

The stall and `count_max` backstops guarantee termination even when the strict du/dtu thresholds are physically unreachable for a given setup (e.g. a transonic wind whose du floors at $\sim 1\%$). Each iteration prints `count`, du , dtu (and the windowed flux-level change when `Level_tol` is set), and the run reports which criterion stopped it. The optional `Force_start` forces the first 1000 iterations regardless (useful when starting near a fixed point). The rationale and history of these settings are in `Update_EXHALE_solver`.

Caveat: du can stop a run prematurely. du measures only the *spatial spread* of $\rho v r^2$ in the wind window; it is blind to a uniform drift of the flux level, and on strongly irradiated planets it *oscillates*, dipping below 10^{-3} transiently long before the wind settles. A WASP-121b reference stopped this way was low by ~ 0.08 dex in $\dot{M}$ with the heating/cooling grossly out of balance (energy residual ~ 30). For production answers use the residual-based stop (`Resid_tol`) or the Newton finish (`Solver: Newton`, §2.5); details in `steady_solver_memo`.

2.5 The steady-state Newton solver (`Solver: Newton`)

The finite-volume steady state can be solved directly — $F(Y) = 0$ for the hydro state, with the ionization equilibrium eliminated locally inside the residual — by a preconditioned Jacobian-free Newton–Krylov method (banded LAPACK preconditioner, frozen WENO weights, diagonal scaling, non-monotone line search; the smooth base valve `Valve`

eps: is required and auto-adopted). This reaches residual levels unattainable by marching at a small fraction of the cost: WASP-121b converges in 8 Newton iterations (~ 200 marching-steps of cost, vs. $>35,000$ insufficient marching steps), HD189733b in 33 s where marching spent >3 h without converging.

Two ways to use it:

- **Production:** add Solver: Newton to input.inp. The run marches (including the automatic two-stage PLM $\rightarrow$ WENO3) until the flux metric du (the radial spread of $\rho v r^2$) drops below the hand-off threshold newton_du_switch (default 10^{-2} ; the du -based convergence stops are suspended), then the Newton solve finishes to Resid tol (default 10^{-3}), and the standard outputs and post-processing follow. The hand-off point is adjustable through an optional third token on the key, Solver: Newton <du_switch> (e.g. Solver: Newton 5e-3 hands off later); a bare Solver: Newton keeps the 10^{-2} default.
- **From an existing state** (recommended for re-converging old runs): put the state in output/*_IC.txt, set Load IC? True, Valve eps: $1.0e-4$, Resid tol: $1.0e-3$, and run with the environment hook

```
ATES_PTC=1 ATES_PTC_JFNK=1 ATES_PTC_DTAU0=1.0 ./EXHALE.x
```

which solves from the loaded state, writes output/, and stops.

The warm start matters: Newton has a finite basin, so the hand-off must wait until the marching warm-up has flattened the wind. The default $du < 10^{-2}$ is calibrated to land inside the basin — handing off earlier (a larger newton_du_switch) risks starting Newton outside it and stalling, whereas a cold-IC start (no warm-up at all) stalls outright. Earlier revisions keyed the hand-off on the residual itself ($\|R\| < 5 \times 10^{-2}$); the du -based key is equivalent in practice and consistent with the flux-based convergence decision (§2.4), while the residual remains the *target* of the JFNK finish. Convergence is linear (frozen-weight refresh and the chemistry noise floor); on planets with a breathing base (HD189733b) the practical floor is $\|R\| \sim 3 \times 10^{-4}$. The Newton finish is also *safe to fail*: the solver tracks the best iterate seen, aborts once no new best residual has appeared for 15 outer iterations, and on failure the run prints JFNK failed, keeps the best iterate, and falls back to plain time-marching with the du -based stops re-armed — an unconverged Newton state is never accepted as the final answer. The associated diagnostic hooks (ATES_RESIDUAL, ATES_JAC_TEST, ATES_NEWTON_TEST, ...) are environment variables, collected in §2.6 (Table 1). Full development record and validation: steady_solver_memo.pdf. Figure 1 shows the production pipeline end-to-end on WASP-121b (Roche): the monitored residual during the marching warm-up (left; note that the du -based stop would have fired long before the residual is actually small) and the JFNK finish (right).

2.6 Environment-variable hooks (diagnostics and expert runs)

A small set of *environment variables* expose diagnostic and expert entry points without touching input.inp (Table 1). They are read once at startup; when unset they have strictly zero effect, so normal runs are unaffected. All of them are diagnostics or validation aids — the only one used in a production workflow is the ATES_PTC family, which implements the “Newton from an existing state” recipe of §2.5 (and examples/04_newton_from_state/). Typical usage:

```
ATES_RESIDUAL=1 ./EXHALE.x # rank a "converged" state
```

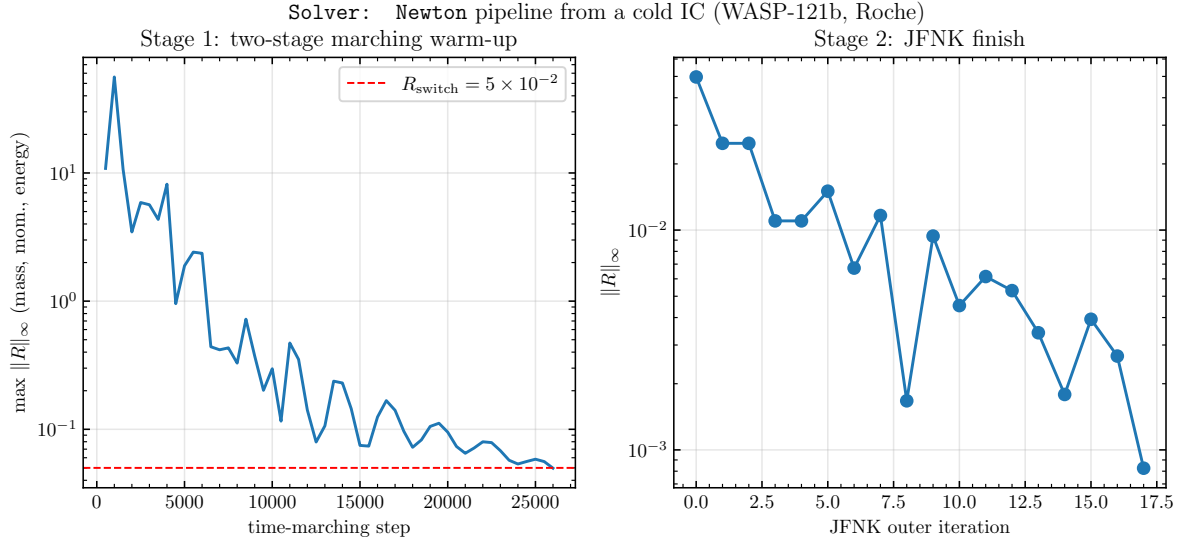


Figure 1: The Solver: Newton pipeline from a cold IC (WASP-121b, Roche). Left: maximum steady residual $\max \|R\|_\infty$ over the mass, momentum, and energy equations during the two-stage marching warm-up; the dashed line is the legacy residual hand-off threshold $R_{\text{switch}} = 5 \times 10^{-2}$ used when this figure was produced (the current default keys the hand-off on $du < 10^{-2}$ instead, at a comparable point on the warm-up). Right: residual vs. JFNK outer iteration during the Newton finish, down to Resid_tol (10^{-3}).

```
ATES_PTC=1 ATES_PTC_JFNK=1 ATES_PTC_DTAU0=1.0 ./EXHALE.x # re-converge it
```

2.7 Metal cooling on/off and the comparison script

Metal-line cooling is active *iff* a `metals.inp` is present (§3.); remove or rename it for a metals-off run. To run the *same* planet both ways automatically:

```
./run_compare_metals.sh # uses ./input.inp and ./metals.inp
```

This compiles once and runs the planet with metals on, then off, saving the two full results to `output_metals_on/` and `output_metals_off/` (each with a copy of the inputs used; the second run reuses the binary). The per-planet notebooks `metals_comparison*.ipynb` read the two folders and plot the temperature, velocity / mass-flux, H and metal ionization, and heating/cooling differences (cf. §7.).

2.8 Wind-AE warm-start initial conditions

A 1-D steady-state Parker-wind solver (a Fortran port of *Wind-AE*; Murray-Clay et al. 2009 / Broome et al. 2025) is bundled under `src/modules/wind_ae/` and can supply the initial condition for a hard-to-launch EXHALE run. There are two ways to use it; the underlying algorithm, ramp, and BC/IC are documented in `docs/wind_ae_solver.tex`.

In-process (recommended): IC mode: windae. Add the single line

Variable	Effect
ATES_DUMP_IC=1	After Load IC? True, dump the restored state to output/IC_dump.txt <i>before</i> the first ionization-equilibrium solve re-equilibrates the species fractions (verifies a snapshot/restart round-trip).
ATES_RESIDUAL=1	Evaluate the steady residual R of the loaded state (WENO3, boundary conditions applied first), print the per-equation norms (mass, momentum, energy), and stop. The criterion-independent way to rank candidate “converged” states (a premature-dip state shows $\ R\ _E \sim O(10)$ despite $du < 10^{-3}$).
ATES_NEWTON_TEST=1	Self-test of the Newton infrastructure (pack_U/unpack_U round-trip and the frozen residual against the ATES_RESIDUAL values); prints and stops.
ATES_JAC_TEST=1	Verify the colored-FD banded Jacobian against a directional finite difference $Jr \simeq [F(Y+\epsilon r) - F(Y)]/\epsilon$; a correct band layout matches to $\sim 10^{-6}$, a wrong one is $O(1)$ off. Prints and stops.
ATES_PTC=1	Solve the <i>loaded</i> state directly with the steady solver (no marching), write output/, and stop. Requires Load IC? True; set Valve eps: $1.0\text{e-}4$ and Resid tol: $1.0\text{e-}3$ (§2.5). Default engine: banded PTC.
ATES_PTC_JFNK=1	With ATES_PTC=1: use the full JFNK solver (GMRES on the matrix-free J) instead of the banded PTC — the recommended engine.
ATES_PTC_DTAU0= $\langle v \rangle$	With ATES_PTC=1: override the initial pseudo-time step $\Delta\tau_0$ (default: the CFL Δt of the loaded state). A large value (e.g. 1.0) probes the pure-Newton regime directly.
ATES_PTC_NFIX= $\langle n \rangle$	With ATES_PTC=1: anchor the first n physical cells (frozen-base experiment; solves the wind on top of a fixed base).
ATES_DIAG_BASE=1	Write output/base_diag.txt (r, n, v, T , heating, cooling of the first few cells over the first steps) to trace IC-startup transients at the base.
ATES_FORCE_HYBRD1=1	Force every ionization-equilibrium solve to use MINPACK hybrd1, skipping the analytic-Jacobian Newton: lets the same binary produce both solvers’ outputs for an A/B validation (Update_EXHALE_solver.pdf §6).

Table 1: Environment-variable hooks. All are read once at startup and have zero effect when unset. The four diagnostic tests (top block) and ATES_PTC runs stop after printing/writing; they do not march.

```
IC mode: windae
```

to `input.inp` (with `Load IC? False`). On startup EXHALE maps the planet’s parameters to Wind-AE, ramps a shipped seed solution to the planet, solves the steady wind, and writes the IC onto the EXHALE grid (`output/*_IC.txt`), which is then loaded automatically — the whole `input → Wind-AE solve → IC → run` sequence is one `./run_EXHALE.sh` (one `EXHALE.x`) invocation. This is a *warm start*: EXHALE re-solves the ionization (and any metals) from the first step, so the Wind-AE IC need not be exactly self-consistent. Always finish with `Solver: Newton` for a quantitative $\dot{M}$ (§2.5). It is best suited to hot Jupiters near the shipped seed; a far-from-seed planet may fail the ramp, in which case the run prints a message advising `IC mode: auto`. The shipped seed and spectrum live in `inputdata/windae_seed.csv` and `inputdata/windae_spectrum.inp`.

Standalone generator: `wind_ae_ic.x`. The same solver builds an IC out of process:

```
make wind_ae_ic      # builds ./wind_ae_ic.x (separate from EXHALE.x)
./wind_ae_ic.x <input.inp> <seed.csv> <spectrum.inp> \
               <IC_dump_grid> <outdir> [rhoscale]
```

It reads an EXHALE `input.inp`, ramps from the seed, and writes `<outdir>/{Hydro_ioniz,Ion_species}_IC.txt` on the grid given by an `IC_dump.txt` from a prior EXHALE run with the same domain settings. Run EXHALE on the result with `Load IC? True + Solver: Newton`. (The standalone tool is independent of the in-process path above; either can be used.)

3. Input files

3.1 `input.inp` — the main parameter file

`input.inp` is read *line by line in a fixed order* for the core block (Table 2), followed by an *optional keyword block* (Table 3) whose lines are matched by keyword and may be omitted (older files without them keep the default behavior). A few lines are *conditional*: the spectrum-property line depends on `Spectrum type`, and the energy-band / X-ray-luminosity lines depend on `Use only EUV?`.

Table 2: `input.inp` core block (fixed order). Conditional lines are marked (*).

Label	Value	Meaning
Planet name:	string	Label for the run (cosmetic).
Log10 lower boundary number density [cm ⁻³]:	real (log ₁₀)	log ₁₀ n_0 , the H number density at the 1 μ bar base.
Planet radius [R _J]:	real (R_J)	Base radius R_p (the 1 μ bar level); the grid radius r is in units of this.

Table 2, continued

Label	Value	Meaning
Planet mass [M_J]:	real (M_J)	Planet mass M_p (sets gravity / the Jeans parameter b_0).
Equilibrium temperature [K]:	real (K)	Base temperature T_0 .
Orbital distance [AU]:	real (AU)	Star–planet separation a (sets the XUV flux $\propto a^{-2}$ and the tidal/Roche potential).
Escape radius [R_p]:	real (R_p)	Inner radius of the constant-momentum convergence window; must lie <i>inside</i> the domain ($< r_{\max}$). For L1-truncated Roche runs use a value below the lobe.
He/H number ratio:	real	Helium abundance $n_{\text{He}}/n_{\text{H}}$; > 0 enables He chemistry.
2D approximate method:	see text	Day–night averaging of the XUV flux and $\dot{M}$: Rate/2 + Mdot/2 (dayside, $\xi = \frac{1}{2}$), Mdot/4 (full-sphere), Mdot, or alpha (then a trailing number sets the rate-correction a_τ).
Parent star mass [M_sun]:	real ($M_\odot$)	Stellar mass $M_\star$ (tidal potential, Roche-lobe size).
Spectrum type:	Power-law Monochromatic Load	Shape of the incident XUV SED.
(*) Power-law index: / Photon energy [eV]: / SED file:	real / real / path	Property of the chosen SED: power-law slope E^{PLind} , the monochromatic energy, or a numerical SED file. With metals, mind the required wavelength coverage (§3.5).
Use only EUV? True/False:	logical	True: EUV band only. False: include X-rays (adds the band’s third energy and an X-ray luminosity line).
(*) [E_low,E_mid,E_high] = [. . .]	reals (eV)	Band edges: two values (EUV only) or three (with X-rays).
(*) Log10 of X-ray luminosity [erg/s]:	real (log ₁₀)	Only if X-rays are included; log ₁₀ L_X .
Log10 of EUV luminosity [erg/s]:	real (log ₁₀)	log ₁₀ L_{EUV} (with a , sets the incident XUV flux).

Table 2, continued

Label	Value	Meaning
Grid type:	Uniform Stretched Mixed	Radial grid spacing. Mixed (fine near the base, stretched outward) is the usual choice.
Numerical flux:	HLLC ROE	Riemann solver for the hydrodynamic fluxes.
Reconstruction scheme:	PLM WENO3	Spatial reconstruction order.
Include He23S?:	logical	Include the metastable He 2 ³ S triplet (needed for the He 10830 line).
Load IC?:	logical	Restart from the previous run's output/*_IC.txt (warm start).
Do only PP:	logical	Skip the time loop and only run the post-processing on existing output.
Force start:	logical	Force the first 1000 iterations regardless of the convergence test (useful when the IC is close to a fixed point).

Table 3: input.inp optional keyword block (any line may be omitted; defaults restore the standard behavior).

Keyword	Value	Meaning
Domain mode:	Spherical	If present and Spherical: pure planetary potential $-b_0/r$, domain extended to Outer radius. Absent (default): full Roche (tidal) potential; outer boundary at the Hill/L1 radius unless Outer radius is given (§1.3).
Outer radius [R_p]:	real > 1 (R_p)	Outer boundary (e.g. 10–15 R_p). In Spherical mode it sets the pure $-b_0/r$ domain; in Roche mode (no Domain mode line) it extends the domain past L_1 while <i>keeping</i> the tidal potential (Yan/Huang-style; pair with IC mode: auto).
Stellar Teff [K]:	real (K)	Stellar T_{eff} for the diluted-blackbody Balmer continuum that photoionizes/heats H($n=2$). Enables the excited-H model when both this <i>and</i> Stellar radius are > 0.

Table 3, continued

Keyword	Value	Meaning
Stellar radius [R_sun]:	real ($M_{\odot}$ -radius)	Stellar radius (sets the Balmer-continuum dilution $R_{\star}/a$).
Jlya RT file:	path	Read an external $\bar{J}_{\text{Ly}\alpha}(r)$ profile to pump H($n=2$) (sets jlya_mode=1).
Jlya escape-prob:	True	Compute $\bar{J}_{\text{Ly}\alpha}$ in-line each step by escape-probability RT (jlya_mode=2).
Stellar Lya flux [erg/cm2/s]:	real	Incident stellar Ly α flux at the planet (the stellar source for jlya_mode=2).
Lya stellar halfwidth [km/s]:	real (km/s)	Stellar Ly α line half-width; controls how deep the trapped stellar beam penetrates (default 70).
Lya stellar boost [-]:	real	Bounded buildup cap for the trapped stellar Ly α beam (default 5).
Deexc heat:	True	Add collisional de-excitation heating of H($n=2$) to the energy budget.
Transonic IC:	True	Use a transonic isothermal-wind initial condition (solved from the potential via the Bernoulli integral) instead of the hydrostatic IC; needed for deep-RLOF bases that do not launch from rest. Auto-enables Force start.
Newton solver:	logical (def. True)	Solve the ionization equilibrium with the analytic-Jacobian Newton method (with a MINPACK hybrd1 fallback); False reverts to the legacy hybrd1. Default is faster ($\sim 25\%$) and reproduces hybrd1 to $\sim 10^{-7}$.
Brent solver:	logical (def. True)	Solve the post-process energy equation with a bracketing (Brent) root-finder that selects the physical root; False reverts to legacy hybrd1 + a $2\times$ -band reject.
du_th [PLM,WENO3]:	two reals	Automatic two-stage reconstruction: run PLM until $du < \text{first value}$ (or a PLM stall), then switch to WENO3 and converge at $du < \text{second value}$ (e.g. 0.5×10^{-3}). Replaces the manual PLM $\rightarrow$ stop $\rightarrow$ Load IC+WENO3 restart of the official ATES recipe.

Table 3, continued

Keyword	Value	Meaning
Stall [tol,N]:	real, int	Stall-detector override (relative du change and window; defaults 10^{-6} , 2000).
CFL:	real (def. 0.6)	CFL-number override.
Hot Parker IC:	real (K)	Warm-seed initial condition: stable hydrostatic density with a warm T /ionization/velocity overlay.
IC mode:	word	Initial-condition family: cold (default), transonic, hot_parker (synonyms for the two keys above), auto — pick automatically from the cold sonic-point topology of the actual potential: an interior sonic point (deep-RLOF via the Roche L1 topology, or low-gravity boil-off, $r_c \simeq b_0/2c_0^2$ inside the domain) selects the transonic IC, otherwise the cold hydrostatic IC — or windae — build the IC in-process with the ported Wind-AE steady-wind solver (§2.8). No tunable threshold for auto; the choice and the escape parameter b_0 are logged. Explicit Transonic IC/Hot Parker IC keys override auto. See docs/auto_ic_design.md and, for windae, docs/wind_ae_solver.tex.
Level tol:	real	Mass-flux <i>level</i> -stability gate: a converged/stalled stop additionally requires the mean $ \rho v r^2 $ over the wind window to change by less than this (relative) across the last N_{stall} steps. Guards against stopping on a transient du dip while the flux level still drifts (≤ 0 = off, default; §2.5).
Resid tol:	real	Residual-based convergence: stop when the steady residual $\max_k \max_j R_{j,k} / \max_j u_k $ falls below the value (monitored every 500 steps), <i>replacing</i> the du test (≤ 0 = off, default). du alone is blind to the energy imbalance (§2.5).

Table 3, continued

Keyword	Value	Meaning
Time stepping:	Local	Per-cell pseudo-time steps (steady-state acceleration; not time-accurate). Reaches the deep state fast but leaves per-cell residuals; pair with a Newton finish.
Energy solver:	Explicit	Revert to the original forward-Euler source update (solver-isolation tests).
Valve eps:	real (code v)	Smooth base valve: ghost velocity $\frac{1}{2}(v + \sqrt{v^2 + \varepsilon^2})$ instead of $\max(v, 0)$. Differentiable at $v = 0$ (required by the Newton solver); the converged answer is insensitive to ε (10^{-4} vs. 10^{-5} tested identical). ≤ 0 = exact legacy valve (default).
Solver:	Newton [real]	Production steady-state solve: marching warm-up until the monitored steady residual $\max \ R\ $ drops below the optional value (default 5×10^{-2}), then the JFNK Newton finish to <code>Resid tol</code> (default 10^{-3}), then the standard final outputs and post-processing. Adopts <code>Valve eps</code> : 10^{-4} automatically if unset. See §2.5.
ATES_photoionization_rate:	True	Revert the He I (1^1S) photoionization cross section to the legacy ATES two-term fit (default: Verner+1996). Does <i>not</i> affect the He 2^3S Penning rate, which is always the temperature-dependent form (§8.).
He_diffusion:	logical (def. False)	Enable diffusive separation of helium: transport the He element ratio $n_{\text{He}}/n_{\text{H}}$ with advection + molecular-diffusion settling, so He/H falls with altitude. Off = He/H frozen at the input He/H number ratio (§3.6).
He_Kzz:	real (cm^2s^{-1} , def. 10^9)	Eddy-diffusion coefficient used with <code>He_diffusion</code> : mixes He/H toward uniform below the homopause (Taylor+2025 K_{zz}).
He_ambipolar:	logical (def. True)	Ambipolar-corrected settling mass — in the ionized wind the polarization field lifts He ions, reducing settling ($\Delta m_{\text{eff}} = 3 - \frac{1}{2}(\bar{Z}_{\text{He}} - \bar{Z}_{\text{H}})$); False = neutral $\Delta m = 3$.

Table 3, continued

Keyword	Value	Meaning
He_alphaT:	real (def. 0)	Thermal-diffusion factor α_T added to the settling drift ($\alpha_T \partial \ln T / \partial r$); 0 = off.
He_metal_diffusion:	logical (def. False)	Also diffuse each trace metal independently against H (own mass and binary D); heavier elements settle more. Off = metals frozen to H (§3.6).

3.2 metals.inp — trace-metal abundances

`metals.inp` is *optional*: if absent, the run is metals-off (H/He only). If present, it lists the total elemental abundance n_X/n_H (summed over ionization stages; the equilibrium solver distributes each element over its stages) for any of C, N, O, Mg, Si, Ca, Na, K, S, Fe. Blank lines and # comments are ignored; any non-zero abundance turns metals on. Four control keys are also accepted: `pp_metals` (metal treatment in the advection post-process: 0=metal-free, 1=frozen at the converged equilibrium (default), 2=re-solve, not yet implemented), `cx_full` (1=full Huang+2023 charge-exchange network; default metal-H only), `cno_cool` (C/N/O line-cooling source: 1=closed-form fits to CHIANTI v11, including NI/NII cooling (default); 0=legacy AIOLOS analytic fits, which have no N cooling and deviate by 40–70% from CHIANTI for O I/O II in the wind-launch region — see `cooling_data/cno_cooling_comparison.ipynb`), and `eos_metals` (bulk-gas metal policy: 1=metals contribute their mass to the gas density and base density, their electrons to the free-electron density and the base pressure correction, and their nuclei to the total particle density — a $\sim 1\%$ mass effect at solar abundances, consistent with the charge balance inside the ionization solver (default); 0=legacy trace approximation, H/He-only gas budget). Solar reference values (Asplund+2009):

```
# metals.inp -- solar n_X/n_H (Asplund+2009)
C 2.69e-4 N 6.76e-5 O 4.90e-4
Mg 3.98e-5 Si 3.24e-5 Ca 2.19e-6
Na 1.74e-6 K 1.07e-7 S 1.32e-5 Fe 3.16e-5
pp_metals 1
```

3.3 opacity.inp — opacity model (optional)

`opacity.inp` selects the bound-free opacity model via `OPACITY_MODEL = A|C|P|T`: **A** analytic (default, the standard ATES cross sections); **C** constant (threshold σ times the per-species factors `OPA_CONST_HI/HEI/HEII/HEITR`); **P** pressure-broadened (Robinson & Catling $f(p) = 1 + a(p/p_0)^n$ via `OPA_PB_FACTOR=a`, `OPA_PB_EXPONENT=n`, `OPA_PB_PIVOT=p_0`); **T** tabulated (per-species two-column .atesopa files, energy [eV] vs. σ [10^{-18} cm^2], given by `OPA_FILE_HI` etc.). Missing keys keep their defaults, so an absent `opacity.inp` is equivalent to model A.

3.4 Excited hydrogen and the Ly α pumping field

The non-LTE H($n=2$) model (the metastable $2s$ + resonance $2p$ levels, Christie+2013) feeds two things: in EXHALE.x the H($n=2$) photoionization adds a Balmer proton source and photoelectric heating to the energy/ionization balance; in TPM.py the $n=2$ population sets the H α /H β transmission. The model is *off by default*; it switches on only when both Stellar Teff and Stellar radius are given (they supply the diluted-blackbody Balmer continuum that ionizes/heats $n=2$), so the standard run is unchanged.

The $2s/2p$ populations are driven by the **Ly α mean intensity** $\bar{J}_{\text{Ly}\alpha}(r)$, which pumps $1s \leftrightarrow 2p$. How $\bar{J}_{\text{Ly}\alpha}$ is obtained is selected by three mutually-exclusive options:

(mode 0) Parameterized — the default with no external input. If neither Jlya RT file nor Jlya escape-prob is present, $\bar{J}_{\text{Ly}\alpha}$ is the Huang et al. (2017, Eq. 6) estimate computed internally from the run’s own state,

$$\bar{J}_{\text{Ly}\alpha}(r) = \frac{0.1 F_{\text{LyC}}}{\Delta\nu_D(r)} \frac{1}{1 + \tau_{\text{Ly}\alpha}(r)},$$

where $\Delta\nu_D = \nu_0 \sqrt{2k_B T / \mu} / c$ is the local Doppler width, $\tau_{\text{Ly}\alpha}(r)$ is the *top-down* line-center Ly α optical depth integrated from the outside in through the H I column, and

$$F_{\text{LyC}} = \xi \frac{10^{L_{\text{EUV}}}}{4\pi a^2} \left(1 - e^{-\sigma_{\text{LyC}} N_{\text{HI}}}\right)$$

is the deposited Lyman-continuum flux (the incident EUV flux times the dayside factor ξ from 2D approximate method and the absorbed fraction set by the neutral-H column N_{HI}). This needs *no* external file — it is a self-contained, approximate “staging” field, adequate to get the coupling and signs right but not a true radiative-transfer solution.

(mode 1) External RT file — Jlya RT file: <path>. Read a tabulated $\bar{J}_{\text{Ly}\alpha}(r)$ profile (two columns, r/R_p and $\bar{J}$ in cgs) produced by a real Ly α radiative-transfer calculation (e.g. a Monte Carlo run); it is interpolated onto the grid and used directly.

(mode 2) In-line escape probability — Jlya escape-prob: True. Compute $\bar{J}_{\text{Ly}\alpha}$ every step from the current state by escape-probability radiative transfer (Neufeld/Harrington wing escape): a trapped *internal* field (recombination + collisional sources closed by the escape probability β) plus a trapped *stellar beam* set by Stellar Lya flux (with the penetration controlled by Lya stellar halfwidth and the buildup capped by Lya stellar boost). This is the most physical of the three but still an approximation; see Update_EXHALE.tex §12 for the full formulae.

TPM.py uses the *same* logic for its H α /H β $n=2$ population: if a Jlya_file is set it is read (mode 1), otherwise the mode-0 parameterized estimate above is used (with F_{LyC} rebuilt from the run’s H I column and the stellar EUV). So a converged run with no Ly α input still yields H α /H β predictions, via the default field.

3.5 Spectral coverage for metal photoionization

Neutral metals have ionization thresholds *far below* the hydrogen Lyman limit (13.6 eV $\equiv$ 912 Å): the lowest is K I at 4.34 eV (2856 Å), followed by Na I, Ca I, Mg I, Fe I, Si I, S I, and C I (Table 4). To photoionize them at all, the modeled

stellar spectrum must therefore reach these long (FUV/NUV) wavelengths — the EUV/X-ray band alone ($\lambda < 912 \text{ \AA}$, $E > 13.6 \text{ eV}$) leaves every neutral metal un-ionized and artificially abundant, corrupting both the ionization balance and the predicted metal lines. How far the spectrum reaches depends on the SED type (Spectrum type in Table 2):

Power-law / monochromatic (Power-law). The code builds its own energy grid. When low-IP metals are active it *automatically* appends a sub-13.6 eV extension (20 log-spaced points) running from the H I edge down to the lowest ionization threshold among the active neutral metals, and evaluates the EUV power law there by *extrapolation* (the EUV normalization is anchored to L_{EUV} , which is defined over $[E_{\text{low}}, E_{\text{mid}}]$, then continued below E_{low}). No extra input is needed — but the FUV/NUV flux that ionizes the metals is just the EUV slope extended downward, which real stars do *not* follow (their flux rises steeply into the NUV). Treat power-law metal ionization as indicative; load a real SED for quantitative metal work. *Caveat:* the He 2^3S -triplet sub-grid and the metal sub-grid are mutually exclusive — a single power-law run cannot carry both.

Loaded SED (Load, SED file:). The file’s own energy/wavelength grid *is* the radiative grid (read verbatim, no extrapolation). The SED file must therefore **extend to at least the lowest threshold of your active metals** — e.g. to $\approx 2860 \text{ \AA}$ if K is included, or to whichever neutral metal in Table 4 has the lowest threshold in the active set. A SED truncated at the Lyman limit ($912 \text{ \AA}$) delivers *zero* ionizing flux to Na I / K I / Ca I / Mg I / Fe I / Si I / C I / S I, so those stages stay spuriously neutral.

Either way, the energy bands set by $[E_{\text{low}}, E_{\text{mid}}, E_{\text{high}}]$ and the luminosities L_{EUV}/L_X still describe the *ionizing* (H/He-driving) part of the spectrum above 13.6 eV; the metal extension is in addition to them.

Table 4: Neutral-metal photoionization thresholds (first ionization stage of each modeled element), as E_{th} and the corresponding wavelength $\lambda_{\text{th}} = hc/E_{\text{th}}$. The stellar SED must extend to at least the longest λ_{th} among the *active* metals to ionize them. Values are the VFKY96/NIST thresholds used in the code (species_table.f90).

Species	E_{th} [eV]	λ_{th} [$\text{\AA}$]	Note
K I	4.34	2856	lowest — sets the grid floor when K is present
Na I	5.14	2413	
Ca I	6.11	2028	
Mg I	7.65	1622	
Fe I	7.90	1569	
Si I	8.15	1521	
S I	10.36	1197	
C I	11.26	1101	
(H I)	13.60	912	Lyman limit (reference)
O I	13.62	910	$\simeq$ Lyman limit
N I	14.53	853	above the limit (already in the EUV band)

3.6 Diffusive separation of helium and metals

By default EXHALE is single-fluid: the composition is re-solved each step by *local* ionization equilibrium, which conserves the element ratios, so $n_{\text{He}}/n_{\text{H}}$ (and each metal/H) stays frozen at the input value at every radius. Setting `He_diffusion: True` adds a genuine transport equation for the helium element ratio $f = n_{\text{He}}/n_{\text{H}}$ — advection at the bulk velocity plus a molecular-diffusion *settling* drift of He relative to H,

$$\Phi_{\text{He}} = -D n_{\text{H}} \left(\frac{\partial f}{\partial r} + f G \right), \quad G = \frac{\Delta m g}{kT}, \quad D = \frac{1.52 \times 10^{18} (1/m_{\text{H}} + 1/m_{\text{He}})^{1/2} T^{1/2}}{n},$$

(Banks & Kockarts 1973), plus eddy diffusion K_{zz} (`He_Kzz`) on the gradient term. Helium then settles out relative to the escaping hydrogen and *He/H declines with altitude*, as in Taylor et al. (2025) and Xing et al. (2023). The equation is solved conservatively (implicit, tridiagonal) and the new split is written back into the composition conserving mass. Physics refinements: `He_ambipolar` (default on) reduces the settling in the ionized wind via the polarization field; `He_alphaT` adds thermal diffusion; `He_metal_diffusion` extends the same operator to each trace metal (heavier elements settle more, so they deplete faster than He).

The strength of the separation scales as (settling speed)/(wind speed): it is suppressed by fast escape and is stronger for heavier species. On a gentle escaper (HD 209458b) helium itself separates and the He 10830 line weakens; on a furious escaper (WASP-121b) helium is dragged out almost unfractionated while the heavy metals still settle strongly. Full physics, derivation, and a two-planet before/after comparison: `docs/design_hehe_diffusion.md` and `docs/version_compare.{md, tex, pdf}`. The flags are **default off**, so standard runs are byte-identical to before.

3.7 Lower-atmosphere connection (Tier 1–3)

Three opt-in layers connect EXHALE’s 1- μ bar base to the atmosphere below (survey, implementation, examples and figures: `docs/lower_atmosphere_coupling.pdf`). All are **default off**.

Tier 1 — analytic lower column (consistency report). Add

Lower column: 1.36 # radius of the 1-bar level [`R_J`], e.g. transit radius

and EXHALE integrates the isothermal- T_{eq} hypsometric column with the chemical-equilibrium $\text{H}_2/\text{H}/\text{He}$ partition (Koskinen et al. 2022) from 1 bar to 1 μ bar and *reports* the derived base radius (equilibrium value + a fully-atomic bracket), the base $\text{H}_2/\text{H}/\text{He}$ fractions and μ , next to the input “Planet radius”. Nothing is overridden. A large base q_{H_2} on a genuinely cool planet signals that the molecular physics below is required.

Tier 2a — molecular base (EOS only). `Molecular base: True` removes the H_2 -bound particles from the base particle budget (`ntot_bc`) using the same equilibrium fit — a lower base pressure and heavier base μ , with the species arrays still atomic. Crude; pair it with Tier 2 for cool planets.

Tier 2 — molecular chemistry.

`Molecular chemistry: True` # H_2 , H_2^+ , H_3^+ , HeH^+ in the coupled equilibrium

`Molecular base: True` # recommended companion (consistent ghost pressure)

adds the four molecular species to the coupled ionization equilibrium (the Koskinen et al. 2022 reaction network), the H_2 photoionization opacity and heating (Yan et al. 1998 cross section), and the optically-thin H_3^+ infrared cooling (Miller et al. 2013 with their non-LTE factor). The output `Ion_species*.txt` gains four columns (H_2 H_2p H_3p HeHp). Requires He; v1 excludes trace metals and `He_diffusion` (the parser refuses the combinations). Caveats: local *equilibrium* chemistry only (no molecular advection), no Lyman–Werner photodissociation, $\gamma = 5/3$ retained. On hot Jupiters the result is a thin molecular base under a sharp $\text{H}_2 \rightarrow \text{H}$ front with an atomic wind above — the atomic assumption becomes a result rather than an input.

Tier 3 — base.inp handoff (and VULCAN). An optional `base.inp` in the run directory overrides the base state before the derived constants are formed (each override is echoed; absent file = no-op):

```
T_base    1140.0    # [K]      -> overrides T0
r_base    1.402     # [R_J]    -> overrides "Planet radius" (1-ubar radius)
HeH_base  0.083     #          -> overrides the He/H number ratio
Kzz_base  1.0e9     # [cm2/s] -> sets He_Kzz (used by He_diffusion)
```

The pre-step can be invoked by *EXHALE itself*: with

```
Lower atmosphere: vulcan 1.36    # or: analytic 1.36 (arg = 1-bar radius R_J)
```

in `input.inp`, the code runs the VULCAN photochemistry (or the analytic column) at startup when no `base.inp` exists, then proceeds with the wind solve on the generated base — subroutine-style, with the first VULCAN run taking hours and later runs reusing the cached result in `<run_dir>/vulcan_work/`. Omitting the key disables the pre-step entirely. The VULCAN/ tree (obtained by `setup_vulcan.sh`; it ships FastChem inside it) carries its own citation requirements — see §3.8. Two generators are provided in `src/utls/`:

- `run_lower.py <run_dir> --r1bar <R_J> [--guillot]` — the analytic column (isothermal T_{eq} , or the Guillot 2010 semi-grey $T(p)$); pure Python, no dependencies beyond numpy.
- `vulcan_to_base.py <out.vul> <run_dir> --mp <M_J> --r1bar <R_J>` — converts a **VULCAN** photochemical-kinetics output into the same `base.inp`, with the *photochemical* H_2/H dissociation state (on HD 189733 b photochemistry dissociates $\sim 11\times$ more H at $1\ \mu\text{bar}$ than chemical equilibrium) and a hypsometric base radius using VULCAN's own $\mu(p)$ and $T(p)$.

Running VULCAN is handled automatically by the driver `src/utls/vulcan_driver.py`, so no manual VULCAN setup is required. When the pre-step is requested it (i) obtains VULCAN if needed (via `src/utls/setup_vulcan.sh`; see §3.8) and copies the EXHALE/VULCAN/ tree into `<run_dir>/vulcan_work/` (once, then reused), (ii) builds a Guillot (2010) semi-grey $T(p)/K_{\text{zz}}$ atmosphere for the planet and selects a stellar UV spectrum from the shipped spectra by the host T_{eff} ($> 6800\text{ K} \rightarrow 51\text{ Eri}$; $5300\text{--}6800\text{ K} \rightarrow \text{solar}$; $4300\text{--}5300\text{ K} \rightarrow \epsilon\text{ Eri}$; cooler $\rightarrow \text{GJ 436}$), (iii) compiles FastChem if its binary is missing, (iv) runs VULCAN to steady state, and (v) converts the result to `base.inp`. The first run takes hours; the converged `<name>-photo.vul` is cached and reused on subsequent runs (regenerate with `--force`). The code root is taken from `EXHALE_ROOT` when set (for relocated installs). VULCAN's networks are H/C/N/O(/S):

it supplies the H_2/H state and molecular mixing ratios, *not* the atomic-metal release (Na/Mg/Ca/Fe) — `metals.inp` stays user-supplied.

3.8 Obtaining VULCAN and FastChem

The lower-atmosphere pre-step uses the **VULCAN** photochemical-kinetics code, which ships the **FastChem** equilibrium-chemistry code inside it. These are third-party open-source codes and are *not* committed to the EXHALE repository (VULCAN/ is git-ignored). Fetch and prepare them with

```
src/utils/setup_vulcan.sh
```

which clones VULCAN into EXHALE/VULCAN/, applies the two modifications EXHALE needs, builds FastChem, and removes stale run products. EXHALE also runs this automatically the first time the `vulcan` pre-step is used and VULCAN/ is missing. Obtain updates from, and *cite*, the original sources:

- **VULCAN** — github.com/exoclime/VULCAN (mirror github.com/shami-EEG/VULCAN). Cite Tsai et al. (2017, ApJS 228, 20) and Tsai et al. (2021, ApJ 923, 264).
- **FastChem** — github.com/NewStrangeWorlds/FastChem (shipped inside VULCAN). Cite Stock et al. (2018, MNRAS 479, 865), and Stock et al. (2022, MNRAS 517, 4070) for FastChem 2.

The modifications `setup_vulcan.sh` applies to the cloned VULCAN tree (i.e. the only changes needed if you set it up by hand):

File (in the VULCAN clone)	Modification
<code>make_chem_funs.py</code>	add <code>encoding=None</code> to the <code>np.genfromtxt(vulcan_cfg.com_file, ...)</code> call so the element-conservation check does not crash on Python 3
<code>vulcan_cfg.py</code>	set <code>use_photo = True</code> and <code>use_live_plot = False</code> (this file is the driver's template)
<code>output/*.vul</code>	delete (run products; regenerated in <code><run_dir>/vulcan_work/</code>)
<code>fastchem_vulcan/</code>	make to build the fastchem binary

On the EXHALE side (in this repository) the pre-step adds `src/utils/setup_vulcan.sh`, `vulcan_driver.py` and `vulcan_to_base.py`, and the `Lower atmosphere: key` in `input_read.f90/parameters.f90`.

4. Output files

All profiles are written to `output/` as plain text, one row per radial cell, in *cgs* units. Two flavors exist: the equilibrium files (`*.txt`, written during the run) and the advection-corrected files (`*_adv.txt`, written once at the end by the post-processor, which re-solves the ionization balance including advection of the wind). The `*_adv.txt` files are the ones the analysis tools read.

Caveat — the `_adv` profile at a “breathing” base. The advection-corrected (`*_adv.txt`) *temperature and ionization* would otherwise develop a spurious spike / sawtooth in the dense lower atmosphere (typically $r \lesssim 1.1\text{--}1.2R_p$ for strongly Roche-filling planets such as WASP-121b). Two compounding causes, both confined to the base, are responsible:

1. *A breathing, inflowing base (a 1-D limitation).* When the planet nearly fills its Roche lobe the 1-D wind is subsonic / only marginally launched at L_1 , so the dense lower atmosphere does not settle into a clean monotonic outflow but into a slowly recirculating state with small *negative* (inflow) velocities and a stagnation point ($v = 0$) near the wind base. This is intrinsic to the spherically-symmetric (1-D) geometry — removing it would require a 2-D/3-D model — and is not a numerical bug.
2. *A spurious hot root in the metal-cooled post-processor.* The post-processor re-solves the per-cell energy/ionization balance *including* advection, assuming an outflow (each cell takes the next-inner cell as its upwind state). In the dense base the non-monotonic metal line-cooling curve gives the energy equation a second, spurious *hot* root that the Newton (MINPACK) solver can land on; through the upwind coupling one bad cell then cascades outward, producing the sawtooth and an overshoot spike (`post_process_adv.f90`). A guard rejects solves that fall outside a band around the converged equilibrium temperature, but mild spikes *within* that band can survive.

The post-processor guards against cause 2 directly: wherever the flow is *not* a clean outflow ($v \leq 0$, the breathing inflow base) it skips the advection correction and keeps the converged equilibrium ionization and temperature (`post_process_adv.f90`). This removes the spurious spike and sawtooth at the source — it breaks the upwind cascade — while preserving the genuine few-percent `_adv` correction in the outflow above the stagnation point. Cause 1 (the breathing base itself) is intrinsic to the 1-D geometry and remains, but it is now represented there by the smooth, physical equilibrium solution. The shipped `_adv` profiles are therefore clean, and the temperature panel of Figure 6 plots the `_adv` profile directly. (Velocity and density are *identical* in the `eq` and `_adv` files — the post-processor corrects only the ionization and temperature.)

4.1 `Hydro_ioniz.txt` / `Hydro_ioniz_adv.txt`

Thermodynamic profiles (7 columns). The dimensionless code variables are restored to cgs with the normalizations n_0 (base density), $v_0 = \sqrt{k_B T_0 / \mu}$, $p_0 = n_0 \mu v_0^2$, T_0 (base temperature), $q_0 = n_0 \mu v_0^3 / R_p$.

Col	Quantity	Unit	Note
1	r	R_p	radius (base = 1)
2	ρ (number density)	cm^{-3}	total density ($= \rho_{\text{code}} n_0$)
3	v	cm s^{-1}	radial velocity ($\times v_0$)
4	p	erg cm^{-3}	pressure ($\times p_0$)
5	T	K	temperature ($\times T_0$)
6	heating	$\text{erg cm}^{-3} \text{ s}^{-1}$	XUV photoionization heating ($\times q_0$)
7	cooling	$\text{erg cm}^{-3} \text{ s}^{-1}$	radiative cooling ($\times q_0$)

4.2 Ion_species.txt / Ion_species_adv.txt

Ion number densities [cm^{-3}] (34 columns). Column 1 is r ; columns 2–7 are the H/He ions; columns 8–34 are the 27 metal-ion stages, in this order:

```
r ; HI HII HeI HeII HeIII HeI* ; CI CII CIII OI OII OIII NI NII NIII MgI MgII MgIII SiI SiII SiIII CaI
CaII CaIII NaI NaII KI KII SI SII FeI FeII FeIII
```

(HeI* is the metastable 2^3S triplet.) The ionization fraction of any element is the stage density divided by the sum over that element's stages. With Molecular chemistry: True four more columns (H2 H2p H3p HeHp, cm^{-3}) are appended after the metals; the # columns header always names the layout.

4.3 Cooling_breakdown.txt

Per-channel radiative cooling [$\text{erg cm}^{-3} \text{s}^{-1}$] vs. radius (written for metals-on runs). Columns: 1= r , 2= T [K], 3= n_e [cm^{-3}], 4=total cooling; 5--8= the four H/He channels (recombination, collisional ionization, collisional excitation, bremsstrahlung); 9--35= the 27 metal-line cooling channels (same ion order as Ion_species). The channels sum to column 4. Useful for identifying the dominant coolant vs. altitude (cf. Huang+2023 Fig. 10).

4.4 Excited_H.txt

H($n=2$) diagnostic (written when the excited-H model is on, i.e. stellar T_{eff} and radius are set). 16 columns: 1 r , 2 T , 3 n_{HI} , 4 n_e , 5 $\bar{J}_{\text{Ly}\alpha}$, 6 n_{2s} , 7 n_{2p} , 8 Balmer proton source S_p , 9 photoelectric heating, 10 de-excitation heating, 11 $\tau_{\text{Ly}\alpha}$, 12--14 ground-state HI photoionization / collisional-ionization / recombination rates (for the proton-budget comparison), 15--16 the internal and stellar contributions to $\bar{J}_{\text{Ly}\alpha}$.

4.5 ATES.out and the mass-loss rate

ATES.out is a human-readable setup report (planet/star parameters, the Jeans parameter, composition, numerics) plus the final result. The **steady-state mass-loss rate** is $\dot{M} = 4\pi\rho v r^2$ evaluated near the outer boundary, corrected by the 2-D-approximation factor from 2D approximate method ($\text{Rate}/2 + \text{Mdot}/2 \Rightarrow \times \frac{1}{2}$, $\text{Mdot}/4 \Rightarrow \times \frac{1}{4}$). It is reported as $\log_{10} \dot{M}$ [g s^{-1}]; divide by M_p and multiply by 1 Gyr to express it in $M_p \text{Gyr}^{-1}$. The helper exhale_io.mdot_log10 / mdot_Mp_per_Gyr reproduce this.

5. Analysis tools

5.1 EXHALE_plots.py — live profile viewer

python3 EXHALE_plots.py [--live N] draws an 8-panel figure from output/: density, velocity, pressure, temperature, momentum $\log_{10}(\rho v r^2)$, the ion densities, and the H and He ionization fractions. With --live N it refreshes every N seconds (watch a run converge). If the *_adv.txt files exist they are overplotted (dashed) against the equilibrium profiles (solid). It also prints $\log_{10} \dot{M}$.

5.2 TPM.py — transmission spectra

TPM.py post-processes a converged run (*_adv.txt + input.inp) into transit transmission spectra and reports the line-center absorption depths (theoretical, instrument-convolved, and instrument+rotation-convolved) for **He 10830**, **Ly α** , **H α /H β** (from the non-LTE $n=2$ population, Christie+2013, pumped by the Ly α field of §3.4), and the metal resonance doublets **Mg II h&k**, **Ca II H&K**, and **Na I D** — the metals get the same full treatment as the H/He lines (both doublet components in one window, theoretical / instrument / +rotation spectra, figures via fig_name_mgii/caii/naI), plus the validated single-component line-center and 4 Å-band depth table; a metals-off run skips them automatically. It supports two geometries via geometry = 'spherical' (Phase 5a) or 'triaxial' (Phase 5b, the 3-D Roche-equipotential reconstruction in roche_recon.py). Set the planet/star parameters in its header (e.g. R_star, rot_period, T_star); these are *not* read from input.inp except R_p , M_p , a , L_{EUV} .

Step-by-step usage. TPM.py reads output/Hydro_ioniz_adv.txt and output/Ion_species_adv.txt from path (default '.'), so it is normally copied into the run directory and executed there:

1. *Ensure the *_adv.txt files exist.* A normal converged run writes them automatically. To post-process an existing state (e.g. a Newton solution produced by the ATES_PTC hook, which writes only the eq files): copy Hydro_ioniz.txt → Hydro_ioniz_IC.txt and likewise for Ion_species, set Load_IC? True and Do only PP: True in input.inp, and run ./EXHALE.x once (seconds).
2. *Edit the header block* of the copied TPM.py: the stellar radius R_star, the rotation period rot_period (use the orbital period for a tidally locked planet), the stellar T_star (enables the Balmer-continuum $n=2$ photoionization; ≤ 0 disables), the instrument resolutions, and the output names fig_name_hei/lya/ha/hb (empty = do not save).
3. *Run* MPLBACKEND=Agg python3 TPM.py (the Agg backend suppresses the interactive windows on headless machines). The script saves the requested figures and prints the line-center absorption depths (theoretical / instrument-convolved / +rotation) and the metal-line summary.

Worked examples (the converged Newton solutions of WASP-121b and HD189733b, with all four H/He spectra and the metal lines): regression/tpm_wasp/ and regression/tpm_hd189/, collected in the notebook regression/steady_solver_results.ipynb. Figure 2 shows the four H/He spectra TPM.py saves for the WASP-121b Newton solution of regression/tpm_wasp/ (run with Include_He23S? True, so the He I 10830 line is populated).

Transit-depth conventions (important). Huang et al. (2023) quote transit depths as the *effective transit radius* R_{eff}/R_* , not a percent absorption; convert from the code's line absorption fraction h via $R_{\text{eff}}/R_* = \sqrt{(R_p/R_*)^2 + h}$. The metal NUV lines (Mg II) are quoted in a 4 Å bin, while the optical lines (Ca II, H α , Na) are line-center; line-center vs. 4 Å can differ by $\sim 3\times$ for an optically-thick line, so always compare in the same definition.

5.3 exhale_io.py — named, physical-unit loaders

examples/exhale_io.py loads every ATES output file into named arrays in physical units, so analysis code need not remember column orders (Table 5). The central object is the Run container: load_run(outdir, inputfile) fills

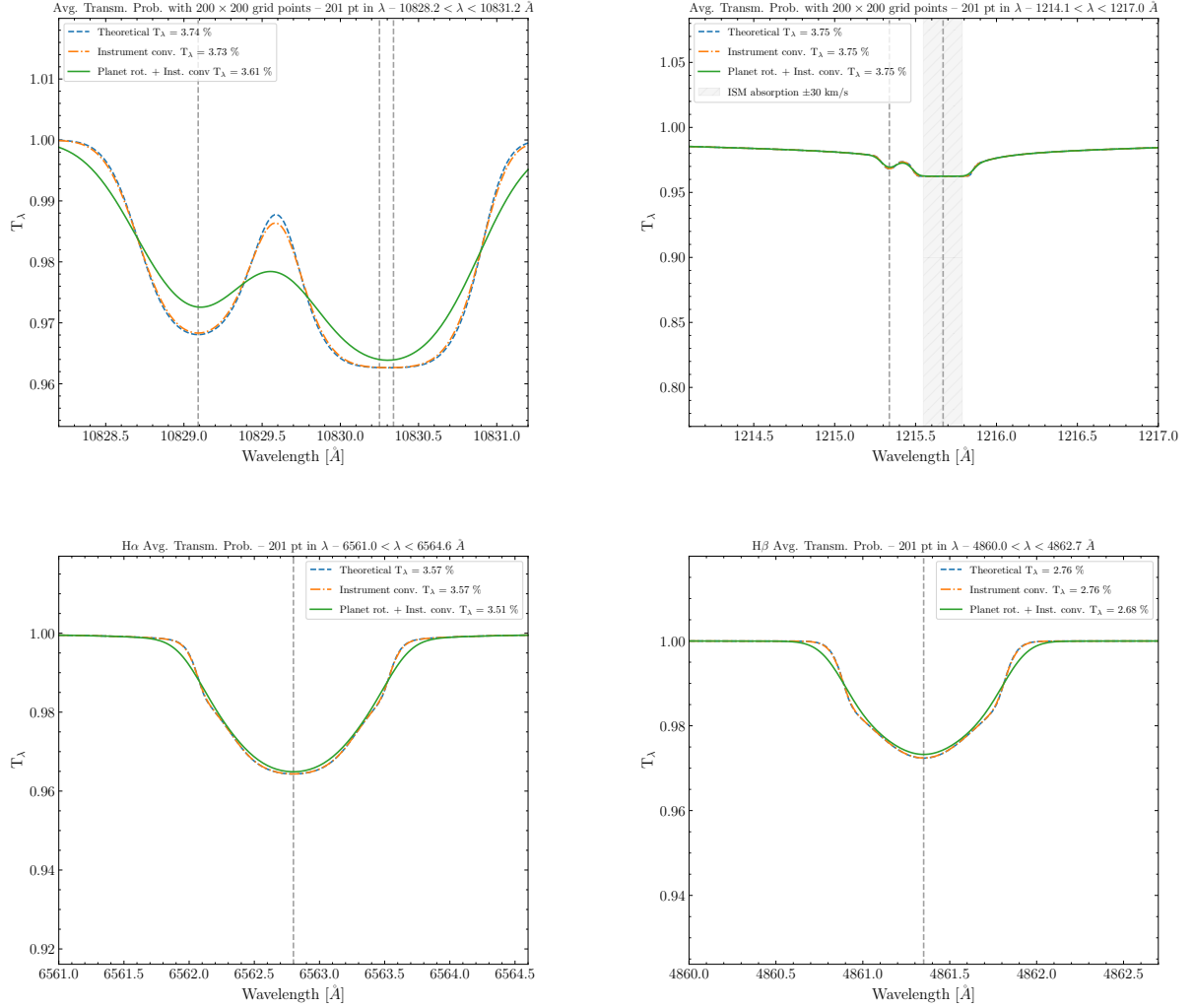


Figure 2: TPM.py output spectra for WASP-121b (Roche, Newton solution with He 2^3S on; regression/tpm_wasp/): He I 10830 (top left), Ly α (top right), H α (bottom left), and H β (bottom right). Each panel shows the theoretical line, the instrument-convolved line, and the instrument+rotation-convolved line as produced by the script.

it with the hydro profiles (r [R_p], n [cm^{-3}], v [cm s^{-1}] with a v_{kms} shortcut, p , T , heat , cool), the full ion-density dictionary `run.ion` (the 6 H/He plus 27 metal stages of §4.), and the parsed input `input.inp` in `run.inp`. By default the `_adv` (advection-corrected) files are read; pass `adv=False` for the equilibrium snapshot. Typical use:

```
import exhale_io as aio
run = aio.load_run('output', 'input.inp')      # _adv profiles by default
print(aio.mdots_log10(run))                    # log10 Mdot [g/s], 2D factor incl.
print(aio.mdots_Mp_per_Gyr(run))              # same in  $M_p/\text{Gyr}$ 
xh = run.x_ion(['HI', 'HII'])                  # stage fractions of one element
plt.plot(run.r, run.T)
plt.plot(run.r, xh['HII'])
cool = aio.load_cooling('output/Cooling_breakdown.txt')
plt.semilogy(cool['r'], cool['chan']['FeII']) # one cooling channel
```

Function	Returns
<code>load_run(outdir, inputfile, adv=True)</code>	a Run: hydro profiles + <code>run.ion</code> dict + parsed input.
<code>Run.x_ion([stages])</code>	ionization fractions of one element, e.g. $x_{\text{ion}}(['\text{HI}', '\text{HII}'])['\text{HII}'] = n_{\text{HII}}/(n_{\text{HI}} + n_{\text{HII}})$.
<code>load_hydro(path)</code>	one <code>Hydro_ioniz(_adv).txt</code> as a dict ($r, n, v, p, T, \text{heat}, \text{cool}$).
<code>load_ions(path)</code>	(r , { <code>ion</code> : density}) from <code>Ion_species(_adv).txt</code> (all 33 stages).
<code>load_cooling(path)</code>	$r, T, n_e, \text{cool_total}$ + chan dict of per-channel cooling (4 H/He channels <code>rec/coll_ion/coll_exc/brems</code> + 27 metal-line channels).
<code>load_excited_H(path)</code>	<code>Excited_H.txt</code> as a dict ($n_{2s}, n_{2p}, J_{\text{lya}}, \tau_{\text{lya}}, \text{Sproton}, \text{Hpe}, \dots$).
<code>read_input(path)</code>	<code>input.inp</code> as a dict ($R_p, RJ, M_p, MJ, a_{\text{AU}}, \text{HeH}, \text{LX}, \text{LEUV}, \dots$ + raw lines).
<code>mdots_log10(run, j_from_top=20)</code>	$\log_{10} \dot{M}$ [g/s]: $4\pi\rho v r^2$ near the outer boundary <i>including</i> the 2D approximate method factor (mirrors <code>EXHALE_main.f90</code> ; see §4.).
<code>mdots_Mp_per_Gyr(run)</code>	the same in units of $M_p \text{ Gyr}^{-1}$.
constants	$RJ, MJ, M_{\text{sun}}, \text{AU}, \text{GYR}$ (cgs).

Table 5: examples/exhale_io.py public interface. Run analysis scripts either from `examples/` or after adding it to the Python path (`sys.path.insert(0, '<repo>/examples')`).

The notebook `examples/EXHALE_analysis.ipynb` uses these loaders to plot the profiles and the parameter-difference comparisons of §6., and `examples/make_figures.py` (which generates every figure in this manual) is a further worked example.

5.4 exhale_to_lart.py — LaRT Ly α input from an EXHALE run

TPM.py estimates the Ly α mean intensity $\bar{J}_{\text{Ly}\alpha}$ with the simple Huang et al. (2017) prescription (§3.4). For a proper treatment of the resonant Ly α radiative transfer that sets the $n=2$ hydrogen population (and hence H α), the converged atmosphere can be post-processed with the Monte Carlo code **LaRT** (Seon & Kim 2020; Seon et al. 2022). examples/exhale_to_lart.py builds a complete LaRT spherical-illumination input set directly from an EXHALE run.

Given a run directory it loads the _adv profiles (via exhale_io.py) and input.inp, and writes

- dens_profile.txt — r [], $n_{\text{HI}} [\text{cm}^{-3}]$ (the Ly α scatterer is ground-state H);
- temp_profile.txt — r [], T [K];
- velo_profile.txt — r [], $v_r [\text{kms}^{-1}]$;
- line_profile.txt — the incident stellar Ly α profile, a double-Gaussian with peaks at $\pm m$ and Gaussian width s (defaults $m = 74$, $s = 49 \text{ kms}^{-1}$, the Huang et al. 2017 / Yan et al. 2022 values; pass --fwhm to read s as FWHM);
- <name>_lya.in — the LaRT namelist (geometry='spherical_atmosphere', source_geometry='stellar_illumination');
- run.sh — a multi-host mpirun launcher.

The length unit is set to (distance2cm = R_p in cm), so all LaRT radii are in and match the EXHALE grid; the star-planet geometry (distance_star_to_planet = a /, stellar_radius = R_* /) is read from input.inp. Because the stellar illumination breaks spherical symmetry, the scattering rate is requested as a cylindrical 2-D grid (geometry_JPa = 2): $P_\alpha(\rho, z)$ is axisymmetric about the star-planet axis, which is the form needed to build the 2-D $n_{2p}(\rho, z)$ population for the H α transit. To populate P_α , build LaRT with make CALCPnew=1 (producing LaRT_calcP.x).

```
# build the LaRT input from a converged EXHALE run, then run it
python examples/exhale_to_lart.py WASP-52b/fxuv0p25_he98 \
    -o WASP-52b/LaRT_lya --name WASP-52b \
    --m 74 --s 49 --rmax 10 --ngrid 201 --nphotons 1e7 \
    --hosts lart4,lart3,lart2
cd WASP-52b/LaRT_lya && ./run.sh
```

It is also importable: import exhale_to_lart as e2l; e2l.build(run_dir, outdir, m_kms=74, s_kms=49, geometry_jpa=2, ...).

6. Worked examples

6.1 A first run: a generic hot Jupiter

The directory examples/tutorial/ contains a minimal, self-contained model of a generic HD 209458b-like hot Jupiter: a spherical domain out to $10R_p$, solar metals, EUV+X-ray power-law SED. The full input.inp is:

```
Planet name: demoHJ
Log10 lower boundary number density [cm^-3]: 12.70
```

```

Planet radius [R_J]: 1.38
Planet mass [M_J]: 0.69
Equilibrium temperature [K]: 1320.0
Orbital distance [AU]: 0.047
Escape radius [R_p]: 2.00
He/H number ratio: 0.083
2D approximate method: Rate/2 + Mdot/2
Parent star mass [M_sun]: 1.13
Spectrum type: Power-law
Power-law index: -1.0
Use only EUV? False
[E_low,E_mid,E_high] = [ 13.60 , 123.98 , 1.24e3 ]
Log10 of X-ray luminosity [erg/s]: 27.0
Log10 of EUV luminosity [erg/s]: 29.30
Grid type: Mixed
Numerical flux: HLLC
Reconstruction scheme: PLM
Include He23S? False
Load IC? False
Do only PP: False
Force start: False
Domain mode: Spherical
Outer radius [R_p]: 10.0
du_th [PLM,WENO3]: 0.5 1.0e-3
Solver: Newton

```

The last two lines enable the automatic two-stage reconstruction and the steady-state Newton finish (§2.5), so the run converges to the true steady state without manual restarts. With `metals.inp` present (solar abundances, §3.), run `./EXHALE.x` in that directory. Figure 3 shows the converged profiles read back with `exhale_io`: the wind heats to $\sim 10^4$ K, the velocity rises monotonically outward (transonic outflow), and hydrogen ionizes across a front near $1.2\text{--}1.4R_p$.

6.2 Turning physics on and off

The same model is re-run with single-line changes to illustrate each option: *metals off* by removing `metals.inp`; *Roche/RLOF* by removing the `Domain mode/Outer radius` lines (the default truncates at $L1$); the *He 10830* line by setting `Include He23S? True`; the *H α /Balmer* physics by adding `Stellar Teff/Stellar radius`; and the *transonic IC* by adding `Transonic IC: True` for a near-unbound (deep-RLOF) base; and *diffusive He/H separation* by adding `He_diffusion: True` (extend it to the metals with `He_metal_diffusion: True`; §3.6). The comparison studies below quantify the effect of the first two.

Tutorial run: generic hot Jupiter (spherical, H/He + metals)

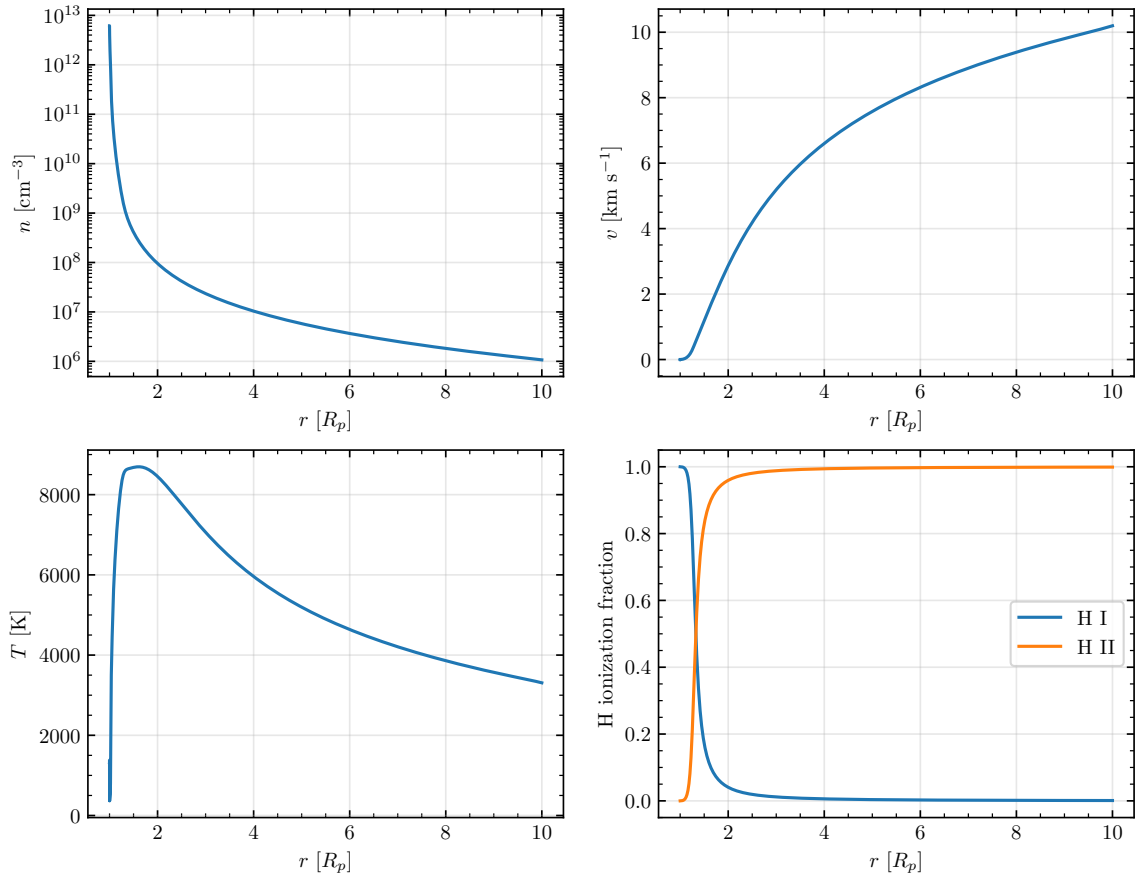


Figure 3: First-run output of the generic hot-Jupiter tutorial: density, velocity, temperature, and the H ionization fraction vs. radius.

6.3 Ready-made example configurations (examples/)

For every solver/physics combination there is a ready-to-run folder under `examples/` (paths relative to the EXHALE root), all for the *same* planet (HD189733b) so that the effect of each option can be isolated. Each folder is self-contained: `cd` into it and run the repo-root binary (`./../EXHALE.x`); outputs land in the folder's own `output/`. Every folder differs from the baseline `01_legacy_marching/input.inp` only by the lines listed in Table 6 (so diff against the baseline shows exactly what an option adds); the ten baseline configurations have been smoke-tested. `11_windae_ic` (HD189733b) exercises the self-consistent C-2 continuation: its Wind-AE ramp converges and writes the IC, though EXHALE's own time-integration of HD189733b is limited by a separate base-breathing instability (§2.8). `12_windae_ic_hd209` is the seed-adjacent HD209458b case.

7. Comparison studies

7.1 Effect of metal-line cooling (metals on vs. off)

Trace metals add strong line cooling (and some photoionization heating and free electrons). Figure 4 compares the tutorial planet with and without `metals.inp`: metal-line cooling lowers the peak temperature by ~ 1800 K (acting as a thermostat that prevents the H/He-only over-heating), while the H ionization structure is barely changed. The effect is far stronger for hotter, more irradiated planets (e.g. WASP-121b, where metal cooling is essential to match the observed temperatures); the detailed per-channel cooling budget is in Figure 5.

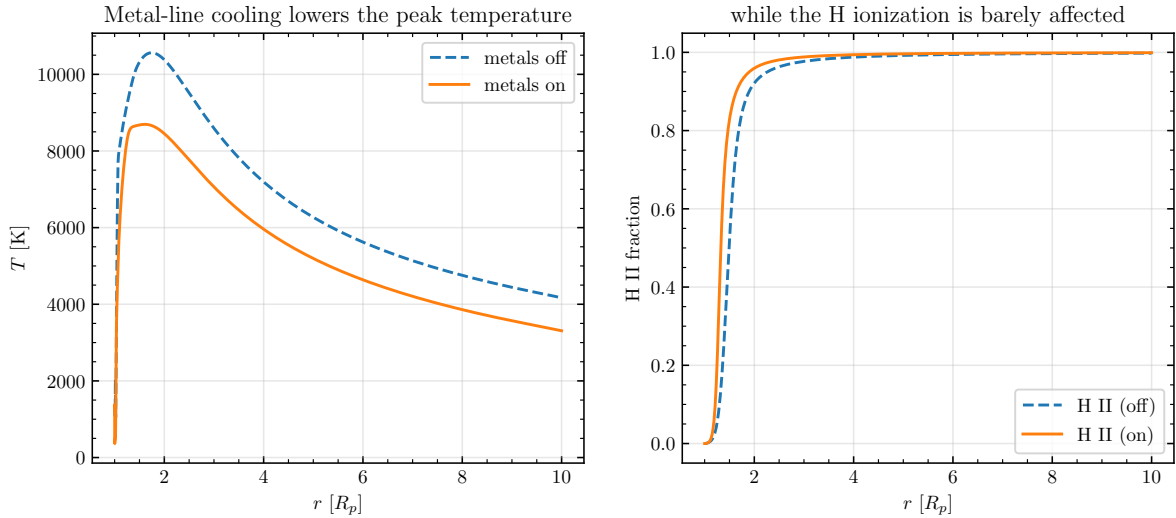


Figure 4: Metals on vs. off for the tutorial planet: metal-line cooling lowers the peak temperature (left) while the H ionization fraction is nearly unchanged (right).

7.2 Spherical vs. Roche (RLOF) and the mass-loss rate

Removing the `Domain` mode: `Spherical` line switches on the full Roche tidal potential, truncated at the L1 radius. Figure 6 compares a spherical model (Case A) and a Roche model (Case B) of WASP-121b: the tidal potential lowers

examples/...	Demonstrates	Added / changed lines
01_legacy_marching	classic ATES v2 baseline: PLM marching, H/He only, Roche domain, <i>du</i> -based stop	(none — baseline)
02_two_stage	automatic two-stage PLM→WENO3 marching (§2.4)	du_th [PLM,WENO3]: 0.5 1.0e-3
03_newton	recommended default: two-stage warm-up + Newton finish (§2.5)	+ Solver: Newton
04_newton_from_state	Newton re-convergence of an existing state (no marching; needs output/*_IC.txt, see the folder README)	Load IC? True, Valve eps: 1.0e-4, Resid tol: 1.0e-3; run via the ATES_PTC hook
05_metals	trace-metal cooling, solar C/N/O/Mg/Ca/Na/Fe (§3.)	metals.inp present
06_he23s	He 2 ³ S metastable level (He I 10830)	Include He23S? True
07_balmer_lya	non-LTE H(<i>n</i> =2) + Ly α pumping (H α /H β , §3.4)	+ Stellar Teff/radius, Deexc heat, Jlya escape-prob, Stellar Lya flux/halfwidth/boost
08_full	everything on (= the HD189733b/planet folder + Newton); feeds TPM.py	07 + metals.inp
09_spherical	spherical domain instead of the default Roche/L1 truncation (§1.3)	Domain mode: Spherical, Outer radius [R_p]: 10.0
10_warm_seed_ic	warm-seed (hot Parker overlay) initial condition	Hot Parker IC: 10000
11_windae_ic	in-process Wind-AE IC for HD189733b (§2.8): far from the seed, so the C-2 continuation (base-BC re-convergence + molecular-layer turn-off) is exercised; the Wind-AE ramp converges and writes the IC. EXHALE's own HD189733b base-breathing instability (separate from the IC) then limits the warm start	+ IC mode: windae, Solver: Newton; symlinks inputdata/ for the shipped seed
12_windae_ic_hd209	in-process Wind-AE warm-start IC that <i>works</i> : HD209458b (not HD189733b) is close to the shipped seed, so the ramp converges and EXHALE warm-starts cleanly (spherical 10 R_p)	HD209458b params + Domain mode: Spherical, IC mode: windae, Solver: Newton; symlinks inputdata/

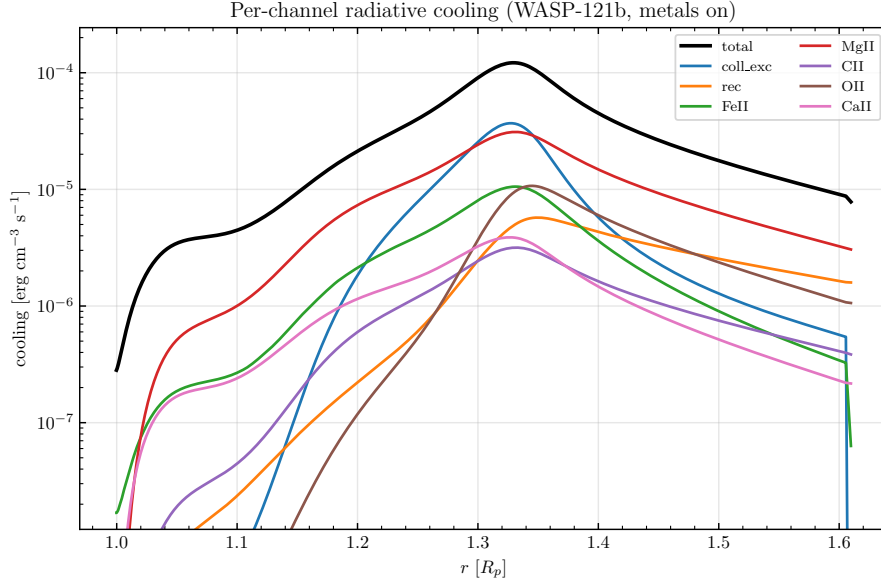


Figure 5: Per-channel radiative cooling vs. radius (WASP-121b, metals on; from `Cooling_breakdown.txt`). Below $\sim 1.4 R_p$ the metal lines (e.g. Fe II, Mg II, C/O) compete with or dominate the H/He collisional channels.

the effective gravity toward the star, so the Roche wind accelerates far faster and the mass-loss rate rises by an order of magnitude ($\dot{M}$ from ~ 0.05 to $\sim 0.34 M_p \text{ Gyr}^{-1}$ here) — the “Roche-lobe overflow” enhancement.

7.3 Transmission spectra

`TPM.py` turns a converged model into transit absorption. The metal resonance lines are the key NUV/optical diagnostics; Figure 7 compares the modelled effective transit radii R_{eff}/R_* (spherical Case A) against Huang et al. (2023). Recall the conventions of §5.2: Mg II is a 4 Å bin, Ca II/Na are line-center, and $R_{\text{eff}}/R_* = \sqrt{(R_p/R_*)^2 + h}$.

8. Atomic and molecular data

EXHALE is a *purely atomic/ionic* code: the modeled species are H, He (including the He 2^3S metastable triplet), and ten trace metals (C, N, O, Mg, Si, Ca, Na, K, S, Fe) across their first two–three ionization stages. There is **no molecular chemistry and no molecular opacity** — no H_2 , H^- , CO, or H_2O — so no molecular line lists or partition functions enter the calculation (the only “molecular” quantity in the code is the mean *molecular* weight μ , i.e. the mean mass per particle). Table 7 collects the provenance of every rate and cross section, grouped by physical process. The H/He baseline is inherited essentially unchanged from the original ATES (Caldiroli et al. 2021, 2022); the metal, excited-hydrogen, and charge-exchange data were added in this work.

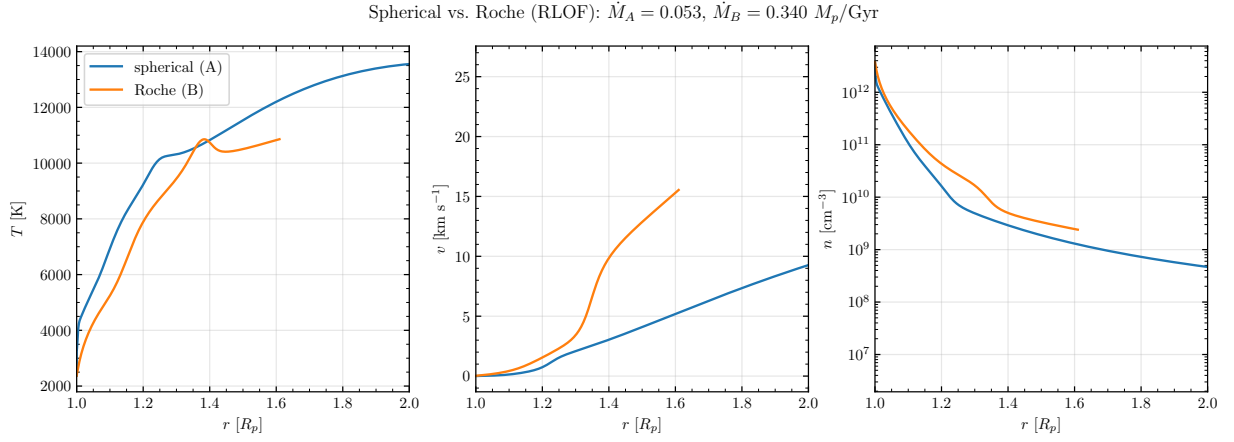


Figure 6: Spherical (Case A) vs. Roche (Case B) WASP-121b: temperature, velocity, and density. The Roche wind reaches L1 ($\approx 1.6R_p$) and accelerates much faster; $\dot{M}$ is annotated. The temperature panel is the `_adv` profile; its base is smooth because the post-processor falls back to the equilibrium solution where the flow is not a clean outflow (see the caveat in §4.). The Roche velocity (middle) dips slightly negative near the base — the genuine, physical “breathing” inflow of a near-Roche-filling planet.

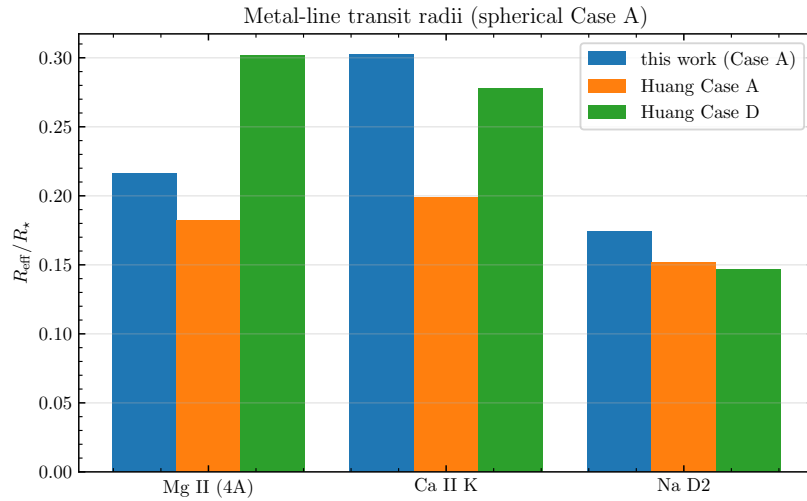


Figure 7: Metal-line transit radii R_{eff}/R_* for spherical Case A, compared with Huang et al. (2023) Case A and Case D.

Table 7: Atomic data sources, by process. Source files: `cross_sec.f90` (cross sections); `Cool_coeff.f90` (collisional rates, recombination, cooling); `charge_exchange.f90`; `excited_hydrogen.f90`; `TPM.py` (transmission line data).

Process	Species	Source
Elemental abundances		
Solar reference	10 metals (<code>metals.inp</code> template)	Asplund et al. (2009)
Helium	He/H ratio	input parameter (He/H number ratio)
Photoionization cross sections		
Bound-free $\sigma(E)$	H I	hydrogenic (Kramers) analytic form (orig. ATES)
	He I	Verner, Ferland, Korista & Yakovlev (1996) default; legacy ATES 2-term fit via <code>ATES_photoionization_rate</code>
	He I 2^3S (metastable)	two VFXY96 (Verner) wings + power-law bridge, fitted to TOPbase/Opacity-Project (extends the orig. Norcross 1971 broken power law past 60 eV)
	C, N, O, Mg, Si, Na, Fe (+ ions)	Verner, Ferland, Korista & Yakovlev (1996), single-shell fits
	K I	Verner & Yakovlev (1995)
Opacity model (runtime)	all (via <code>opacity.inp</code>)	A=analytic above; P=pressure-broadened (Robinson & Catling); T=tabulated files
Collisional (electron-impact) ionization		
Rate $C(T)$	H I, He I, He II	analytic fits (orig. ATES)
	C–Fe, all stages	Voronov (1997, ADNDT 65, 1)
Recombination (radiative RR + dielectronic DR)		
Case B (RR)	H II, He II, He III	Hui & Gnedin (1997) fits (DR negligible)
Metastable	He II $\rightarrow 2^3S, 1^1S$	power-law fits (orig. ATES)
Penning ionization	He $2^3S + H$	Taylor et al. (2025) temperature-dependent fit (replaces the const. 5×10^{-10} ; always on)
RR + DR	C, N, O, Mg, Si, Na, K, S, Ca II (+ stages)	Badnell (2006, ApJS 167, 334) RR + Badnell adf48 DR (TAMOC <code>clist_K</code>)
RR only	Ca I (Ca II+ e)	Shull & Van Steenberg (1982); no DR
RR + DR	Fe I, Fe II	Huang et al. (2023) analytic fit (validation; Badnell switch deferred)

Table 7, continued

Process	Species	Source
Charge exchange (charge transfer)		
metal + H/H ⁺ (default)	23 reactions (Group A)	Huang et al. (2023, ApJ 951, 123) Table 4
full network (cx_full)	+ He+H, metal+He, metal+metal	Huang et al. (2023) Table 4 (Groups B/C/D)
Radiative cooling		
recomb., coll. ion./exc., free-free	H, He	orig. ATES (Hui & Gnedin 1997; Black 1981; Gaunt-factor bremsstrahlung)
metal line cooling (analytic)	C I/II, N I/II, O I/II	closed-form fits to CHIANTI v11.0.2 (default); legacy AIOLOS fits via cno_cool 0 (no N cool- ing)
metal line cooling (analytic)	Mg I/II, Ca II H&K, Na I D, Fe II coronal	closed-form fits to CHIANTI v11.0.2 (Burgess- Tully descaled; 1–3% accuracy); Na I D via effec- tive $\bar{g} \approx 0.2$
metal line cooling (tabulated)	Fe I; Fe II (T, n_e)	NIST f -values + Van Regemorter; CHIANTI mul- tilevel statistical equilibrium
fine-structure (opt., use_2lev_cool)	[O I] etc.	Lique et al. (2017) / Draine (2011)
Excited hydrogen H($n=2$) and Lyα		
2s/2p rate equilibrium	H($n=2$)	Christie, Arras & Li (2013) Table 2
recombination cascade to $n=2$	H	Draine (2011)
Ly α pumping field (mode 0)	H	Huang et al. (2017) Eq. 6
Transmission line data (TPM.py)		
λ , f -value, A_{ki}	He 10830, Ly α , H α , H β , Mg II, Ca II H&K, Na I D	NIST ASD
line profile	all	Voigt (Faddeeva, scipy.special.wofz)

Note on Fe. Iron recombination currently uses the Huang et al. (2023) analytic RR+DR fit (chosen to validate against Huang’s Fig. 12) while every other metal uses the Badnell tabulation; unifying Fe onto Badnell is a documented deferred item. **Note on completeness.** The metal grid carries the stages needed for the XUV-driven ionization balance and the observed NUV/optical lines; higher stages and additional elements (e.g. the full Fe forest, K I, Mg I) beyond those tabulated above are not modeled.